

Ontrack Task3.1P Start Activity

Injection attacks in web applications occur when an attacker exploits vulnerabilities to inject malicious code or commands into an application's input fields, which are then executed by the application's interpreter or database. In this workshop we will see what vulnerabilities in web applications allows us to enter unintended input values. The popular Cross-Site scripting attacks (XSS) attacks are based on this idea to use vulnerable web applications to target victims and obtain sensitive information about them.

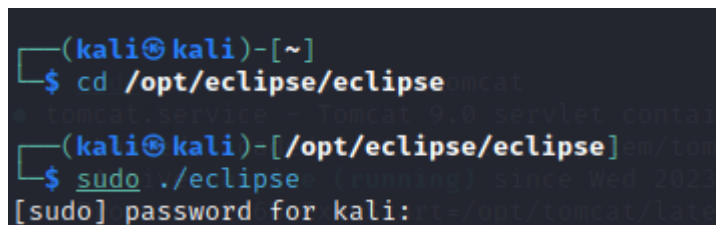
We will test few methods of injecting malicious input in our spring-mvc based web application.

Activities:

1. Start eclipse using the command

```
cd /opt/eclipse/eclipse
```

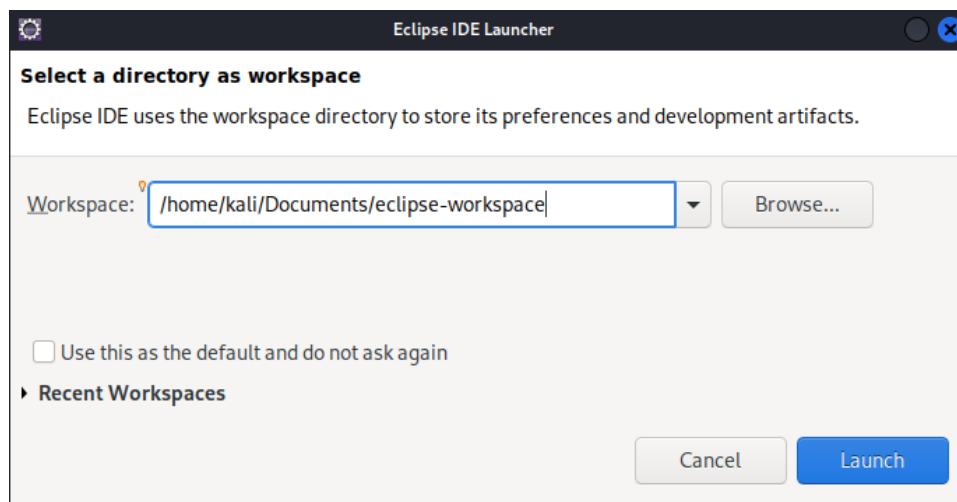
```
sudo ./eclipse
```



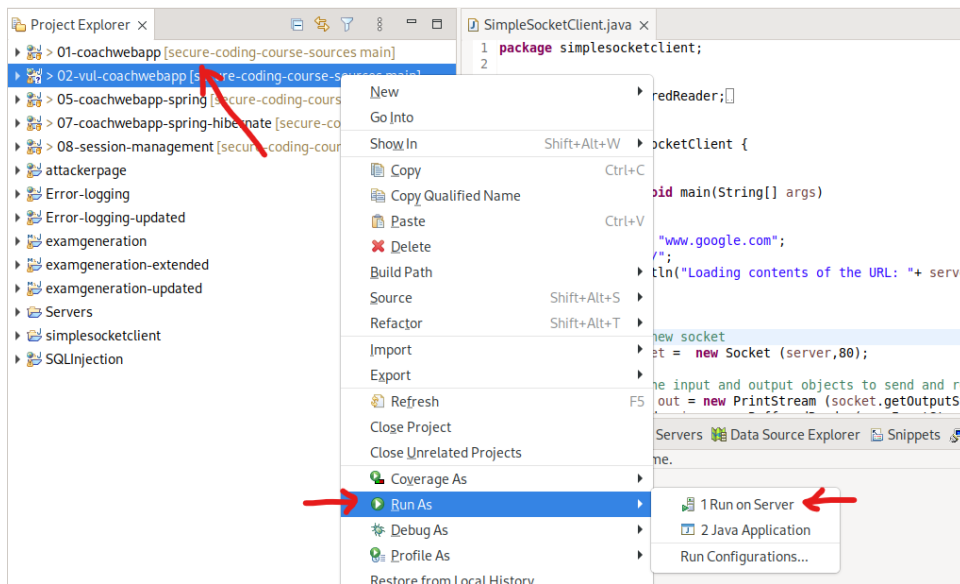
```
(kali㉿kali)-[~]  
$ cd /opt/eclipse/eclipse  
  
(kali㉿kali)-[/opt/eclipse/eclipse]  
$ sudo ./eclipse  
[sudo] password for kali:
```

when prompted for password enter "kali"

2. Click on **Launch** in the Eclipse IDE Launcher.

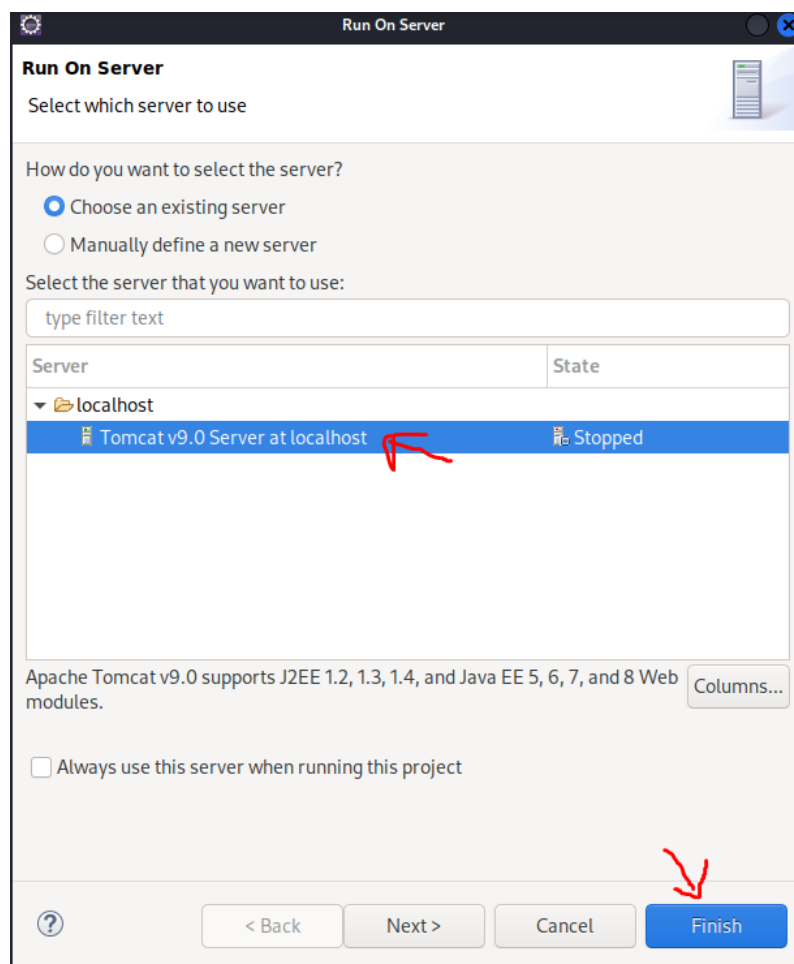


From the project explorer access the 02-vul-coachwebapp application and right click on it



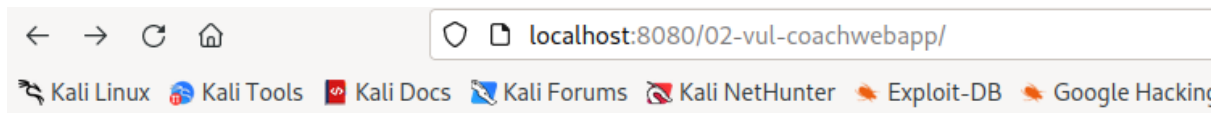
Select **Run As** and choose **Run on Server**

On the Run-on Server screen, select the Apache Tomcat server and click on Finish.



You will notice the webserver loading and the following web app will be opened in the web browser.

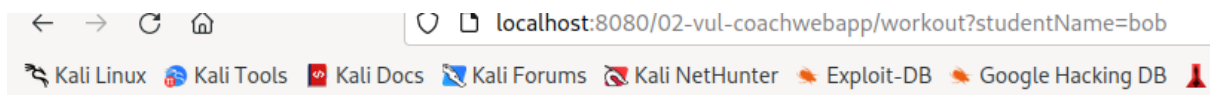
<http://localhost:8080/02-vul-coachwebapp/>



Workout consultation with experts

Enter bob in the name field.

Click submit and you will see the below response from the server.



Coach's Message

Hi bob!

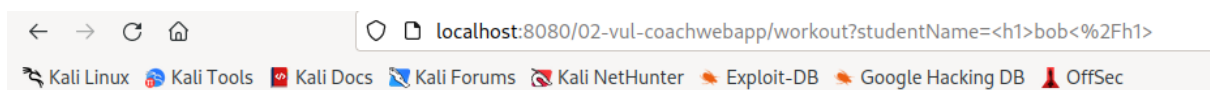
No training today

Now let's enter something not expected by the server and see the response from the server

Click on the back button and go back to the query screen and enter the below value in the name field and click on submit query.

<h1>bob</h1>

Notice the response from the server. You just did a command injection.



Coach's Message

Hi

bob

!

No training today

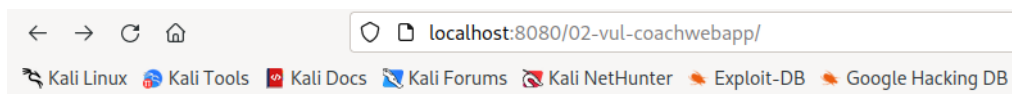
The value you entered just now was a HTML code. The server inserts that input into the coach's message to the user and is displayed with HTML formatting by the web browser.

3.

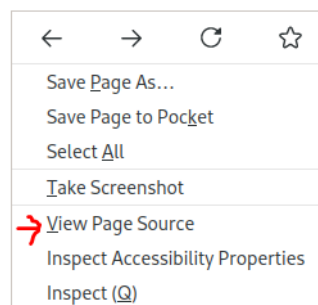
URL generation

Cross-Site Scripting (XSS) is a type of security vulnerability that occurs when an attacker injects malicious scripts into web pages viewed by other users. These scripts can be executed by the victim's browser, potentially allowing the attacker to steal sensitive information, perform unauthorized actions, or deliver malware. For this to work the attacker will generate a URL with the inserted code and share it with the victims in the form of phishing emails or embedding it in attacker-controlled web sites and luring the victim to access it.

Now we will try to generate a URL with the injected code which can be shared with others. To achieve this, we need to look at the variable associated with the input field and generate a GET request URL. On the home page of the vulnerable web app right click and select "view page source"



Workout consultation with experts

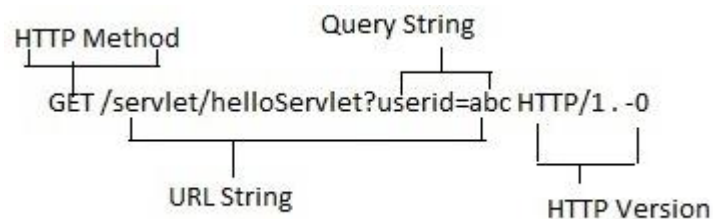


```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="ISO-8859-1">
5 <title>SIT218 Secure Coding</title>
6 </head>
7 <body>
8 <h1>Workout consultation with experts
9 <br>
10
11 <br></h1>
12
13
14
15 <form action = "workout" method = "Get">
16 <input type = "text" name = "studentName" placeholder="What's your name?"/>
17 <input type = "submit"/>
18 </form>
19
20 </body>
21 </html>

```

As you can see, this webpage contains a HTML form with a variable “studentName” and the GET request is sent to fetch the workout page. The correct way to craft a URL based on this is to use the correct path i.e “workout” and pass the values to the variable “studentName” and these are separated using the “?”. Below is an example of a GET request with variables separated by the “?”.



Read more about the HTTP request methods here:

https://www.w3schools.com/tags/ref_httpmethods.asp

Based on this the final URL with the injected code will be as below:

<http://localhost:8080/02-vul-coachwebapp/workout?studentName=<h1>bob</h1>>

Copy paste this URL in the browser, and you will see the response from the server as obtained previously.

You can also enter a multiline HTML code such as

<http://localhost:8080/02-vul-coachwebapp/workout?studentName=<h1>Welcome!</h1><p>click on the below link to download a custom workout plan</p>>

Inserting Scripts

Now try inserting the below javascript into the field and observe the response. You just

<script>alert("You have been hacked!");</script>

You just injected a script into the webpage. You can take this further and achieve the DOM based XSS. Document Object Model is web browser's hierarchical representation of elements of a web page. Using these DOMs, the adversary can trigger unexpected changes on the client side resulting in client-side attack. DOM based cross site scripting occurs when frontend code accepts a user's input (source) and passes that input to another function that displays the results back to the page (sink) in an unsafe manner.

Insert the below script into the name field and view the result in the browser.

```
<script>document.getElementsByTagName("h1")[0].innerHTML="Hacked"</script>
```

4. Try more XSS payloads from here <https://github.com/payload-box/xss-payload-list>

5. Analyse the code to understand the vulnerability

This is an example of a Java web application that displays user comments on a webpage. The application allows users to submit comments, and those comments are displayed on the website without proper validation and sanitization.

```
@WebServlet("/comments")
public class CommentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        String comment = request.getParameter("comment");
        String htmlResponse = "<p>" + comment + "</p>"; // Vulnerable line
        response.getWriter().write(htmlResponse);
    }
}
```

Discuss in the class how the highlighted line is vulnerable to XSS attacks.